

# Arman Hassan

516-373-1256 | armanmansoorhassan@gmail.com | linkedin.com/in/arman-hassan1 | github.com/amansoor

## EDUCATION

### University of North Carolina at Chapel Hill

Chapel Hill, NC

Bachelor of Science in Computer Science — Minor in Data Science

August 2023 – May 2027

- Coursework: Data Structures, Algorithms, Systems, Computer Organization, Linear Algebra, Models of Computation, Machine Learning, Intro to AI, Mobile Computing, Hardware Security, Data Management

## TECHNICAL SKILLS

**Languages:** Java, Python, JavaScript, TypeScript, C, SQL, HTML/CSS

**Frameworks:** React, React Native, Next.js, Node.js, Expo, FastAPI, Tailwind CSS

**Cloud & DevOps:** AWS (Bedrock, S3, IAM), Vercel, Docker, Git, GitHub

**Databases:** PostgreSQL, pgvector, Redis, Amazon S3 Vectors

**AI & Data:** OpenAI API, Claude, RAG, LLM Agents, scikit-learn, pandas, NumPy, Streamlit

**Spoken Languages:** English (Native), Urdu (Native), Punjabi (Native), Hindi (Proficient)

## RELEVANT EXPERIENCE

### Timing

Remote

Software Engineering Intern

January 2026 – Present

- Built an **LLM agent (OpenAI API)** that ranks LinkedIn contacts by priority, cutting manual input by **60%**
- Built a **RAG** pipeline with **pgvector** over **10,000+** contacts to generate personalized outreach drafts
- Cut API latency from **~450ms** to **under 90ms** with **PostgreSQL** indexing and **Redis** caching

### Introduction to Programming

Chapel Hill, NC

Teaching Assistant

August 2025 – December 2025

- Supported **200+ students** in introductory Python through weekly office hours and lab sessions, debugging code and clarifying core concepts like control flow, functions, OOP, and memory diagrams.
- Graded assignments and collaborated with course staff on feedback to improve student performance.

### Vogro (Stanford-affiliated)

Remote

Software Engineering Intern

June 2025 – August 2025

- Built **Python** tools that automated repetitive data tasks, with clear logging and error handling so issues were easy to find and fix, cutting manual processing time by **40%**.
- Created and connected **APIs** using **FastAPI** and **Node.js** that let the **React** website, backend services, and outside tools share data, helping ship features in **3+** releases used by **500+** people.
- Made pages load **20%** faster by reducing unnecessary API calls and cleaning up UI components, so the site ran smoothly for elderly users on older, slower devices.

## PROJECTS

### Degree Planner AI

Chapel Hill, NC

Personal Project | Next.js, TypeScript, AWS Bedrock, S3 Vectors, Claude, Vercel

September 2026

- Built an AI chatbot on **Vercel** that helps UNC students plan coursework across popular majors and minors
- Built a **RAG** pipeline on **AWS Bedrock** using **Titan** embeddings and **S3 Vectors** to ground **Claude's** answers
- Added an LLM routing layer that identifies each student's program and filters off-topic queries early
- Implemented cost and access controls, including rate limits, an access code, and an automated **AWS** budget cap

### Industry Resilience Predictor

Chapel Hill, NC

Carolina Data Challenge Hackathon | Python, pandas, scikit-learn, Streamlit, GitHub, Docker

September 2025

- Built a **machine learning** pipeline to predict how **90+** industries recovered from COVID-era disruptions
- Cleaned and merged **10,000+** time-series data points with **pandas**, handling gaps and misaligned dates
- Trained a **scikit-learn** regression model on this data to estimate each industry's drawdown and recovery trends
- Built an interactive **Streamlit** dashboard to compare industries and visualize predicted recovery metrics
- Used **Docker** and **Git** to keep environments consistent and streamline collaboration across the team

### Arcade-Style Space Battle Game

Chapel Hill, NC

Hack110 Hackathon — **1st Place** | Python, Pygame, OOP, Git

April 2024

- Won **1st place** at Hack110 by shipping a full Pygame arcade shooter end-to-end in under 24 hours.
- Built core game systems including collision detection, enemy spawning, and progressive difficulty scaling.